EECS 348 Group One

<Project Name>
Software Requirements Specifications

Version 1.5

Revision History

Date	Version	Description	Author
12/03/2026	1.0	Section 2 Draft	Hudson Rupp
13/03/2026	1.1	Section 2 Completed	Hudson Rupp
14/03/2026	1.2	Section 3 Draft	Jacob Harmsen
14/03/2026	1.3	Section 1 Completed	Logan
14/03/2026	1.4	Section 3 Completed	Jacob Harmsen
15/03/2026	1.5	Reviewed the document	Jacob Harmsen

Table of Contents

1. Introduction 4
   1.1 Purpose 4
   1.2 Scope 4
   1.3 Definitions, Acronyms, and Abbreviations 4
   1.4 References 4
   1.5 Overview 4

2. Overall Description 5
   2.1 Product perspective 5
      2.1.1 System Interfaces 5
      2.1.2 User Interfaces 5
      2.1.3 Hardware Interfaces 5
      2.1.4 Software Interfaces 5
      2.1.5 Communication Interfaces 5
      2.1.6 Memory Constraints 5
      2.1.7 Operations 5
   2.2 Product functions 5
   2.3 User characteristics 5
   2.4 Constraints 5
   2.5 Assumptions and dependencies 5
   2.6 Requirements subsets 5

3. Specific Requirements 5
   3.1 Functionality 5
      3.1.1 <Functional Requirement One> 6
   3.2 Use-Case Specifications 6
   3.3 Supplementary Requirements 6

4. Classification of Functional Requirements 6

5. Appendices 6

Software Requirements Specifications

1. Introduction

This document is an outlining of the project which can be used as an outlining. A guide for the development of the code, either by a small group of developers or the whole team. In this document, you can look back on what the overall goal is for the project, alongside describing how the software should work and how the consumer can expect to interact with the product. The function and description of how the project should work is the overall point of this document.

1.1 Purpose

The purpose the project is to create a working math problem solver, which can take in math problems and interpret what the problems symbols and characters are then use the proper order to solve it in. So, it should be able to read a math problem like (5+5)-7 * 5 and be able to simplify it down to 10 – 35 then -25. Showing that it uses PEMDAS and understands the order of operations. Design constraints are very light as they must be made in C or in C++ and no other language. It also should be able to handle, and symbols required by the rubric and longer problems than just a few numbers.

1.2 Scope

The scope we have is to have a functioning program that can completely solve any math problem that involves any symbols in PEMDAS. So, no higher-level math like calculus will be required or built in. The uses are for any basic arithmetic to be solved quickly. It should also be free to develop since it's a small group with no external help or resources and takes a couple months for the whole project to be completed.

1.3 Definitions, Acronyms, and Abbreviations

Definitions

- C/C++ is the language that will be used and is considered a more powerful language.
- Functionality is the specific hoops and checks that need to happen before and after the math is done to make sure the math problems input are good and won't cause errors.
- Constraints are anything that must be used or implemented in the project like language or taking parenthesis into account in a problem.
- Interfaces are ways for systems to communicate or users to communicate with the systems.
- Use cases are how users can interact with a product or system.

1.4 References

No references or outside sources have been used in this document.

1.5 Overview

The simple overview of this document is to outline the requirements of the project and set a guideline to follow when we begin to design and plan out the actual development of the code. It's organized into two more major sections, the first is the Overall Requirements which covers the basic functions and guidelines

for the coding section of the project like number types used, language, and any constraints for the project.

The second section is Specific Requirements which gets into much more detail on what certain outputs would be and how the answer will be displayed or errors should be handled.

2. Overall Description

2.1 Product perspective

The product acts as the arithmetic parser and evaluator intended to be a part of a larger compiler, although on release, it is required to be standalone and communicate through a command-line interface.

2.1.1 System Interfaces

Eventually, the program will need to interface with the broader language compiler. This means that there must be clean input functions and return values that can be easily decoupled from the initial command-line interface. Documentation in the code should also be provided so that the implementation into the larger compiler can be easily completed by non-team members.

2.1.2 User Interfaces

The program must have a user-friendly command-line interface where users can manually type arithmetic expressions and receive the final calculated value. If the user inputs an invalid expression, invalid characters, or does not follow the input format, the program should handle these errors gracefully and display clear and informative error messages to the terminal.

2.1.3 Hardware Interfaces

This program doesn't have any direct hardware interfaces as its input and output are handled through the command-line which is managed by the operating system, and it does not need any hardware peripherals to evaluate the arithmetic expression. However, it should be ensured that the program runs correctly on the KU EECS cycle servers.

2.1.4 Software Interfaces

This program will be written in C++ entirely and will primarily utilize the C++ Standard Template Library for foundational containers and algorithms, and iostream for the command-line interface. Other C++ standard libraries will likely be used depending on the specific data types implemented.

2.1.5 Communication Interfaces

This program runs locally and on its own and does not interface with any communication or network protocols.

2.1.6 Memory Constraints

While no specific memory constraints are defined, the program should be free of memory leaks by correctly handling memory allocation and should generally avoid excessive memory use if possible.

2.1.7 Operations

Normal operation of this program consists of the user compiling and running the program, inputting an expression into the command-line interface prompt, and viewing the displayed calculated result or an error message if an error occurs followed by another prompt or termination.

2.2 Product functions

The program must implement a function to first tokenize the input expression and store it in a data structure that represents the structure of the expression so that it can be easily evaluated. The evaluation must handle addition, subtraction, multiplication, division, modulus, exponentiation, and unary operators in the expression. The evaluation must also follow the standard precedence of operators: parenthesis, exponents, multiplication and division, and addition and subtraction. Parenthesis should be correctly identified, and the expressions inside such parenthesis should be handled in the correct order. The program should also handle associativity with addition, subtraction, multiplication, division, and modulus handled from left-to-right and exponentiation handled right-to-left. The

program must also correctly handle numeric constants which for the initial release is only integers but may require accommodating floating point values in subsequent releases. The program will handle all errors gracefully including incorrect input format as well as errors within the expression itself such as division by zero.

2.3 User characteristics

Because this program will be interfaced using a command-line interface, and because of the nature of this program, it is assumed that the user will have general computer science knowledge. It is most likely that the user will be a developer, engineer, or student. The interface with the user should show clear and informative error messages, and documentation should assume general computer science knowledge rather than provide overly verbose explanations of how to use the program.

2.4 Constraints

The program must be developed in C++ and utilize Object-Oriented Programming principles in its structure, including in the data types used to represent the expression and easily handle evaluation. Although the program is intended to be used as part of a larger compiler in the future, the program must initially handle input and output through the command-line interface.

2.5 Assumptions and dependencies

It is assumed that upon initial release that the program will support only integers for numerical constants as well as the default operator set of addition, subtraction, multiplication, division, modulus, exponentiation, unary operators, and parenthesis. It is also assumed that the standard C++ Standard Template Library will be used as well as iostream for command-line input and output. Other C++ standard libraries may be used if needed.

2.6 Requirements subsets

After the program has been completed, there may be future changes requested. Although the program is required to handle input and output through the command-line interface, this will likely need to be removed in the future after it is integrated with the full language compiler. The initial version of the program will only support integer numeric constants, but the accommodation of floating-point input values may be requested in the future.

3. Specific Requirements

The Arithmetic Expression Evaluator system must allow users to input arithmetic expressions, validate the input, and compute the correct result following standard operator precedence rules. This section describes the functional requirements for processing and evaluating expressions and any additional requirements such as system constraints, performance, and usability.

3.1 Functionality

The functional requirements help with defining the system's functions and behaviors. It must support several key features and describe what the system does when it comes to interacting with the user and arithmetic expressions,

3.1.1 <Input Arithmetic Expression>

The system must allow the user to input an arithmetic expression which might contain operators, parentheses, numeric constants, integers, and floats.

3.1.2 <Expression Validation>

The system needs to validate the input expression before performing the arithmetic. If the input is invalid, the system will output an error message.

3.1.3 <Display messages>

The system needs to display commands and the output of the arithmetic.

3.1.4 <Error handling>

The system should be able handle any error without terminating unexpectedly

3.2 Use-Case Specifications

Actor: User

The user inputs an arithmetic expression and receives the results. The system should validate then evaluates the expression. After doing so, it should display the results or error messages.

1. The user enters an arithmetic expression
2. The system validates the expression
3. The system evaluates the expressions according to the operator rules
4. The system displays the result or an error message

3.3 Supplementary Requirements

The Arithmetic expression Evaluator system shall operate through a command line interface that allows users to input arithmetic expressions and receive results directly in the terminal. The system shall process expressions efficiently so that results are displayed immediately after evaluation under normal usage conditions.

4. Classification of Functional Requirements

Functionality	Type
Expression input	Essential
Expression validation	Essential
Display messages	Essential
Error handling	Essential
Floating Point Numbers	Desirable

5. Appendices

The appendices included in this document are provided for reference purposes only and are not considered part of the formal requirements.

5.1 Operator Precedence Table

Precedence	Operator	Associativity
1	( ) Parentheses	N/A

2	^ Exponentiation	Right to left
3	*,/,% Multiply, Divide, Modulus	Left to right
4	+,- Addition, Subtraction	Left to right